

ASSIGNMENT NO. 1

Java Environment Setup and Basic Program Execution

Name of Student	Aryan Karn
PRN	25070122254
Semester / Year	III Semester, AY 2026-27
Subject	Programming with Java
Assignment No.	1
Title	Java Environment Setup and Basic Program Execution

1. Problem Statement

Develop an application based on the Java environment setup (JDK, JRE, JVM), variables, methods, and constructors, and demonstrate the implementation of these concepts through a suitable example program.

2. Learning Objectives

- Understand the theoretical concepts of the Java environment (JDK/JRE/JVM) and core language building blocks (variables, methods, constructors).
- Implement these concepts practically in a working Java program.
- Analyse and interpret the program's output.
- Apply the concepts to solve practical, real-world programming problems.

3. Theory

3.1 JDK, JRE, and JVM

JDK (Java Development Kit) is the complete software package used to develop Java applications. It contains the JRE plus development tools such as the compiler (javac), debugger, and other utilities needed to write and build Java programs.

JRE (Java Runtime Environment) provides everything required to run a compiled Java program. It contains the JVM together with the core class libraries (java.lang, java.util, etc.) and supporting files, but it does not include development tools like the compiler.

JVM (Java Virtual Machine) is the engine that actually executes Java bytecode (the .class file produced by the compiler). It loads, verifies, and runs the bytecode, converting it into machine-specific instructions at run time. The JVM is what makes Java platform-independent, since the same bytecode can run on any operating system that has a compatible JVM.

Relationship: $JDK \supset JRE \supset JVM$. In short, you write code, javac (part of the JDK) compiles it into bytecode, and the JVM (inside the JRE) executes that bytecode.

3.2 Variables

A variable is a named memory location used to store data that can change during program execution. Java variables have a data type, a name, and a scope. In the accompanying program, `studentName` (String), `prn` (String), and `marks` (double) are instance variables, while `collegeName` is a static variable shared by every object of the class.

3.3 Methods

A method is a block of code that performs a specific task and can be called (invoked) whenever that task needs to be executed. Methods can accept parameters, may or may not return a value, and help break a program into smaller, reusable, and manageable units. `calculateGrade()` is an example of a method that returns a value (String), while `displayDetails()` is a void method that performs an action (printing) without returning anything.

3.4 Constructors

A constructor is a special block of code, with the same name as its class, that is automatically invoked when an object of that class is created. It is used to initialise the object's instance variables. A default constructor takes no parameters and assigns default values, whereas a parameterized constructor accepts arguments and uses them to initialise the object with specific values at the time of creation.

4. Program

File name: JavaBasicsDemo.java

```
public class JavaBasicsDemo {

    // ----- Instance variables (different data types) -----
    String studentName; // String type
    String prn;         // String type (PRN kept as String, it can be long)
    double marks;       // double (floating-point) type

    // ----- Static variable (shared across all objects) -----
    static String collegeName = "XYZ Institute of Technology";

    // ----- Default constructor -----
    public JavaBasicsDemo() {
        studentName = "Unknown";
        prn = "00000000";
        marks = 0.0;
    }

    // ----- Parameterized constructor -----
    public JavaBasicsDemo(String studentName, String prn, double marks) {
        this.studentName = studentName;
        this.prn = prn;
        this.marks = marks;
    }

    // ----- Method WITH a return type -----
    public String calculateGrade() {
        if (marks >= 75) return "Distinction";
        elseif (marks >= 60) return "First Class";
    }
}
```

```

    elseif (marks >= 40) return "Pass Class";
    else return "Fail";
}

// ----- Method with NO return type (void) -----
public void displayDetails() {
    System.out.println("College Name : " + collegeName);
    System.out.println("Student Name : " + studentName);
    System.out.println("PRN Number : " + prn);
    System.out.println("Marks Obtained: " + marks);
    System.out.println("Grade : " + calculateGrade());
    System.out.println("-----");
}

// ----- main(): entry point executed by the JVM -----
public static void main(String[] args) {

    System.out.println("==== Java Environment Details =====");
    System.out.println("JDK Version: " + System.getProperty("java.version"));
    System.out.println("JVM Name : " + System.getProperty("java.vm.name"));
    System.out.println("JVM Vendor : " + System.getProperty("java.vendor"));
    System.out.println();

    // Object 1 - created using the DEFAULT constructor
    JavaBasicsDemo obj1 = new JavaBasicsDemo();
    System.out.println("Object created using Default Constructor:");
    obj1.displayDetails();

    // Object 2 - created using the PARAMETERIZED constructor
    JavaBasicsDemo obj2 = new JavaBasicsDemo("Aryan Karn", "25070122254", 82.5);
    System.out.println("Object created using Parameterized Constructor:");
    obj2.displayDetails();

    // Object 3 - shows the class being reused for a second student
    JavaBasicsDemo obj3 = new JavaBasicsDemo("Rohan Sharma", "25070122100", 55.0);
    System.out.println("Object created using Parameterized Constructor:");
    obj3.displayDetails();
}
}

```

5. Output

(The exact JDK version/vendor lines will reflect the JDK installed on the machine used to run the program; a representative run looks like this.)

```

==== Java Environment Details =====
JDK Version: 21.0.10          (shown for the JDK installed on your system)
JVM Name : OpenJDK 64-Bit Server VM
JVM Vendor : Private Build (or Oracle Corporation / Eclipse Adoptium)

Object created using Default Constructor:
College Name : XYZ Institute of Technology
Student Name : Unknown
PRN Number : 0000000

```

Marks Obtained: 0.0

Grade : Fail

Object created using Parameterized Constructor:

CollegeName : XYZ Institute of Technology

StudentName : Aryan Karn

PRN Number : 25070122254

Marks Obtained: 82.5

Grade : Distinction

Object created using Parameterized Constructor:

CollegeName : XYZ Institute of Technology

StudentName : Rohan Sharma

PRN Number : 25070122100

Marks Obtained: 55.0

Grade : Pass Class

6. Conclusion

Through this assignment, the Java environment setup and the roles of the JDK, JRE, and JVM were studied and understood. A Java program was successfully designed and executed that demonstrates instance and static variables, methods with and without return values, and both default and parameterized constructors. The output confirms that constructors correctly initialise object state, and that methods correctly process and return data based on that state. This strengthens the understanding of Java as a platform-independent, object-oriented language and builds a foundation for more advanced object-oriented programming concepts.

7. Questions

Q1. Explain the main concept used in Java environment setup, variables and methods.

The Java environment setup revolves around three components: the JDK, which is used to write and compile programs; the JRE, which supplies the libraries and support files needed to run programs; and the JVM, which actually executes the compiled bytecode and gives Java its platform independence. Once the environment is set up, a program is built using variables, which are named storage locations that hold data of a particular type (such as int, double, or String), and methods, which are named blocks of code that group together a sequence of statements so that a task can be performed on demand, optionally taking inputs (parameters) and optionally producing an output (a return value). Together, these concepts let a programmer store data meaningfully and organise the logic that operates on that data into clear, reusable units.

Q2. What are the advantages of using Java variables and methods?

- Reusability: a method can be called from multiple places instead of duplicating the same code.
- Modularity: complex problems can be broken into smaller, manageable methods, making the program easier to design, read, and maintain.
- Data storage and manipulation: variables let a program store, update, and reuse values throughout its execution, rather than hard-coding data everywhere it is needed.

- Abstraction: a method hides its internal implementation and exposes only what is needed (its name, parameters, and return type), so it can be used without knowing how it works internally.
- Easier debugging and testing: since logic is separated into small units, errors are easier to isolate and each method can be tested independently.
- Type safety: Java variables are strongly typed, which helps catch data-type errors at compile time rather than at runtime.

Q3. How can this implementation be improved?

- Add input validation, for example checking that marks fall within the valid 0-100 range before assigning them in the constructor.
- Use encapsulation properly by making instance variables private and exposing them only through getter/setter methods, rather than accessing them directly.
- Accept data interactively using the Scanner class so that details are entered by the user at run time instead of being hard-coded.
- Add exception handling (try-catch) to gracefully manage invalid input or unexpected run-time errors.
- Introduce method overloading to provide multiple constructors or grade-calculation methods for different use cases.
- Store multiple student records in a collection (such as an ArrayList) and use loops to display and process them all together.

Q4. What are the real-world applications?

- Student/Employee management systems that store records and compute derived results such as grades, salaries, or attendance percentages, exactly as demonstrated by this program.
- Banking applications, where constructors initialise account objects (account number, holder name, balance) and methods handle deposits, withdrawals, and interest calculations.
- E-commerce platforms, where product and order objects are created through constructors and methods compute totals, discounts, and taxes.
- Enterprise software of any kind, since the JDK/JRE/JVM setup is the same foundation used to build and run large-scale Java applications, including Android apps (which use a similar compile-and-run model) and backend services built with frameworks like Spring.